Code Inspection Report

PenUltimate: IWAC Conference App

Version 1.0

Created By Thread Zeppelin: Mariia Maksymenko, Grace Murray, Noah Rushing, Tess Nunes, and Benjamin Miller

IWAC Conference App
Code Inspection Report

Table of Contents

Page

PenUltimate: IWAC Conference App

1. Introduction
    1.1 Purpose of This Document
    References
    1.2 Coding and Commenting Conventions
    1.3 Defect Checklist
2. Code Inspection Process
    2.1 Description
    2.2 Impressions of the Process
    2.3 Inspection Meetings
3. Modules Inspected
    3.1 Navigation and Application Layout Module
    3.2 Authentication Module
    3.3 Agenda and Session Management Module
    3.4 Messaging Module
    3.5 Profile Management Module
    3.6 Notifications Module
4. Defects

Appendix A – Team Review Sign-off

Appendix B – Document Contributions

1. Introduction

We are performing a Code Inspection for PenUltimate who are creating a conference application for IWAC. "The client for this project is part of the Association for Writing Across the Curriculum (AWAC), which organizes the International Writing Across the Curriculum Conference (IWAC), a multi-day event that hosts both educators and researchers. In 2027, UMaine is hosting this conference. The client is interested in having a mobile conference application that can be repurposed for future conferences. The client noted having issues with last year's application, such as having to scroll through too many events and being unable to filter through them (Whova, 2025). The client also mentioned that when a previous app was used on mobile devices, it opened up a web page that was not optimal (LineUpr, 2025). The problem that the client is trying to solve is finding an effective way to allow in-person attendees to navigate the session schedule and find the info they need, as well as creating a way for virtual attendees to have more of an interactive conference experience."

1.1 Purpose of This Document

The purpose of this document is to create a written report of the code inspection that has been performed by Thread Zeppelin of PenUltimate's IWAC Conference Application. Through this, we will have an understanding of PenUltimate's coding and commenting conventions; defects; along with a walkthrough of the code inspection process. This process will provide a third-party perspective allowing PenUltimate to have time to correct any errors that have occurred.

References

PenUltimate. (2025). *Software Requirement Specifications (SRS) for the IWAC Conference Application for Heather Falconer* (Version 1.0). University of Maine.

PenUltimate. (2025). *Software Design Document (SDD) for the IWAC Conference Application for Heather Falconer* (Version 1.0). University of Maine.

PenUltimate (2025). IWAC Conference App. GitHub. https://github.com/umaine-capstone-iwac/IWAC-Conference-App

LineUpr. (2025). The event application solution to boost your event communication – LineUpr . LineUpr GmbH. https://lineupr.com/en

Whova. (2025). Whova: Award-winning event apps and event management . Whova, Inc. https://whova.com/

Options Prettier. (2017). Prettier.io. https://prettier.io/docs/options

1.2 Coding and Commenting Conventions

PenUltimate is currently coding primarily in TypeScript with only 4% being JavaScript. The JavaScript is used to create jest testing configuration, eslint configuration, and a reset-project document.

PenUltimate has set up linting within their GitHub repository workflow using prettierrc and ESLint. These tools are standard when creating coding in TypeScript. These allowed PenUltimate to specify rules and enforce them throughout the entire code. In their settings.json file, their default formatter is prettier-vscode. They have a total of eight conventions enforced by prettierrc: "semi": true, which adds semicolons to the end of statements; "singleQuote": true, which ensures all string literals are formatted with single quote "trailingComma": "all", which prints trailing commas wherever possible and allows trailing commas in type parameters; "printWidth": 80, specifies the line length that can be used before being wrapped (80 is the recommended length); "tabWidth": 2, specifies the number of spaces per indentation-level; "arrowParens": "always", allows includes parentheses around a single arrow function parameter; "bracketSpacing": true, prints spaces between brackets in object literals; "endOfLine": "lf" ensures the files only contain Linux-style line endings. PenUltimate implemented an action on save to automatically fix formatting errors. Their ESLint is configured to use recommended rules and throws an error when prettierrc formatting is not up to par.

PenUltimate used a developing method that reduced the necessity of comments until recently. Initially, they divided the work where everyone worked on coding different screens. They also did not do any pull requests or code reviews during this time. Within the last four weeks they have committed to have at least one comment per function and one comment per database query. There does not seem to be a convention on whether the comments should be above, below, or next to the line it is describing and may do multiple ways in one file (Figure 1) (Figure 2).

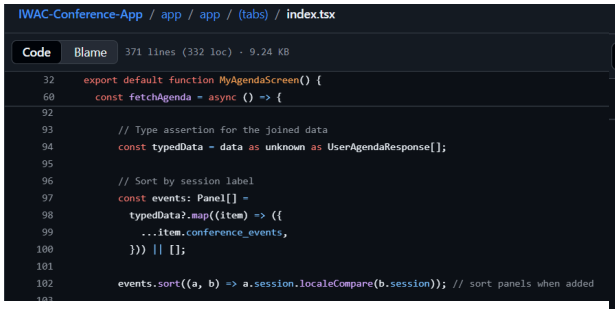

Figure 1

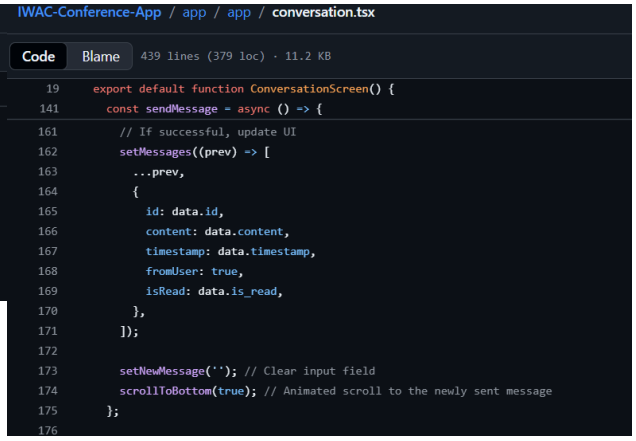

Figure 2

1.3 Defect Checklist

The following checklist was used during the inspection of PenUltimate's IWAC Conference app. It served as a guide during the inspection process, and not every item resulted in a confirmed defect. Each one represented an area reviewed for possible weaknesses in correctness, maintainability, security, usability, and standards compliance.

Category	Possible defect	What we should check for
Coding conventions	Inconsistent naming conventions	Check whether variables, functions, components, screens follow a consistent naming style and an outside reviewer can clearly understand their purpose.
Coding conventions	Unused files, assets, or imports	Check for presence of unused images, scripts, components, or imports that take up storage and add confusion.
Coding conventions	Inconsistent formatting	Check for violations of ESLint/Prettier rules such as spacing, semicolons, quote style, line length, or indentation.
Coding conventions (TypeScript specific)	Weak typing or overuse of any	Check for missing type safety, weak interfaces, or props/state that are not strongly typed.
Commenting	Missing comments	Check for functions without comments where the purpose or behavior is not obvious. Additionally check for database queries without comments explaining purpose or expected results.
Commenting	Inconsistent comment placement or style	Check for comments placed above, beside, or below code inconsistently,

		since it could reduce readability and make the codebase look unprofessional.
Commenting	Outdated or misleading comments	Check for comments that no longer match the implementation or describe old behavior.
Correctness / Logic Errors	Requirements not fully implemented	Check for features described in requirements that are incomplete, missing, or only partially working.
Correctness / Logic Errors	Incorrect conditional logic	Check for branching or validation logic that allows invalid states and/or fails valid ones.
Correctness / Logic Errors	Incorrect state updates	Check for react state changes that may produce incorrect UI, incorrect rendering, or inconsistent values.
Correctness / Logic Errors	Navigation or redirect errors	Check for incorrect routing, broken redirects, or navigation flows that only work on one platform.
Data Validation	Missing input validation	Check for user inputs that are not being checked for empty values, invalid formats, excessive length, or unsafe content.
Data Validation	Missing required-field enforcement	Check for fields required by the specification but not enforced in the UI or database workflow.
Security Oversights	Overly permissive user actions	Check if users are able to access, message, modify, or view data beyond what is intended.

Security Oversights	Inadequate content moderation	Check if there are safeguards against inappropriate uploads, offensive bios, or unsafe user-submitted text.
Security Oversights	Unsafe database interaction	Check if they are trusting Supabase defaults without confirming row-level security, authorization boundaries, and some kind of query protections.
Error Handling	Missing error handling	Check if database, authentication, upload, or fetch failures are not clearly handled.
Error Handling	Poor user-facing error messages	Check for errors that are vague, misleading, or not actionable for the user.
User Friendliness	Confusing labels or prompts	Check for buttons, text, or UI labels that do not match the action being performed.
User Friendliness	Inconsistent terminology	Check for mixed use of terms that are confusing to the user.
Testing	Insufficient automated test coverage	Check if there are no meaningful unit/integration tests for important logic.
Testing	Difficult-to-test architecture	Check if there is tight coupling between UI, navigation, and data access that makes isolated testing difficult.
Reusability	Hard-coded conference-specific behavior	Check for logic or content that would make reuse for future years difficult.
Reusability	Limited support for historical conference data	Check for design choices that may prevent access to previous years’

		presentations or content.

2. Code Inspection Process

Code inspection is a process any software product must undergo during the software development life cycle. It's an important step to ensure that all requirements have been met, coding conventions are being followed, and there are no bugs present. This process is typically done after a prototype has been created, during the code and unit testing phase. It can be done either by the team that created the product or by a third party.

The current state of PenUltimate's code shows the project has almost completed all the requirements agreed upon in October of 2025. They assured us in the beginning of CIR meeting all requirements have been met except the forgot password functionality that was currently being implemented. The goals of our code inspection are to double check the requirements have been met and check for any major errors; extraneous code lines and files; ensure security and safety baselines are being followed; and help to assure software quality.

2.1 Description

The code inspection for the IWAC Conference App was conducted as an outside review by the Thread Zeppelin team. The goal of the inspection was to understand the structure of the application, evaluate coding, and identify potential defects related to functionality, security, testing, and maintainability. The inspection began with Thread Zeppelin reading through PenUltimate GitHub and asking questions as they arose. When questions were asked about features in the application PenUltimate was able to pull up their app and demonstrated the major functional components to help answer these questions.

The application is built using React Native with Expo Router for navigation and Supabase for authentication and database management. During the inspection meeting, the development team guided the reviewers through the project structure and major modules. These included the navigation layout, authentication system, session agenda system, messaging system, profile management features, and notification settings. The inspection team reviewed the repository structure, which contains directories for application routes, reusable components, utility functions, and testing files. The team also discussed how the application handles user authentication, session filtering, messaging between conference participants, and profile updates.

Rather than following a formal step-by-step inspection method, the process consisted primarily of an interactive collaboration with question-driven review. The Thread Zeppelin team asked questions related to coding conventions, security practices, testing procedures, and deployment readiness while examining the repository and code files.

Particular attention was given to areas such as authentication flows, data validation, testing coverage, and commenting practices. Notes were taken throughout the meeting to document observed issues, possible defects, and design decisions explained by the development team. Although the inspection process was less formal than a traditional structured inspection, it allowed the team to gain a clear understanding of the implementation and identify several potential defects and improvement opportunities.

2.2 Impressions of the Process

Our impression of the code inspection process was likely quite different from that of the other teams. Since we had decided to pivot our project in a new direction, the inspection became an opportunity to reflect on our timeline and the mistakes we made throughout our capstone journey. Overall, we found this to be a very effective experience for our team.

Having a third party walk through our timeline and ask questions about our thought process made it clear that we should have voiced our concerns earlier and sought guidance from our professor and TA sooner than we did. That realization was one of the most valuable takeaways from the entire process as early-stage software engineers, learning to communicate concerns early and often is a lesson that will carry forward well beyond this course.

On the positive side, participating in the code inspection alongside our sister team was genuinely beneficial. We walked through the main modules of their project together, asking questions about the implementation details along the way. This included things like how they handled account creation and authentication, how their messaging system worked between users, and how data was stored and managed across the different events the app would be hosting. Seeing how they approached their project, how they divided responsibilities, and what systems they chose for storing data, usernames, and passwords gave us a useful outside perspective and new ideas to apply going forward. If we were to do this process again, we would want to spend more time going deeper into the specifics of their code, such as how certain components were architected, why particular design decisions were made, and how they handled edge cases in areas like authentication and messaging.

The two modular units we identified as having the lowest likelihood of remaining flaws were the Supabase client setup (app/lib/supabase.ts) and the Notifications fetch logic (app/app/(tabs)/notifications.tsx). The Supabase client is a single-responsibility file that simply initializes a client using environment variables, there is very little logic to get wrong, and its narrow scope makes it easy to audit at a glance. The notifications tab, while not the most feature-rich part of the app, has a clear and straightforward data flow: fetch user notifications, join with the notifications table, and map the results into a simple structure. With no complex state management or user input to handle, the surface area for bugs is small. These units benefit from doing one thing and doing it simply, which is generally a good indicator of reliability.

The modular unit we considered most likely to still contain flaws was the messaging system, split across messages.tsx and conversation.tsx. The preview list logic in messages.tsx involves multiple dependent queries, fetching messages, deriving unique partner IDs, then fetching user and profile data for each partner, creating several points where data could be missing, mis-ordered, or stale. Our inspection noted concerns about sorting by timestamp and unread message handling, both of which are easy to get subtly wrong. conversation.tsx raised a separate concern: there did not appear to be any real-time update mechanism, meaning a user would not see new messages without manually refreshing. In a messaging feature, that is a significant usability flaw that could also mask deeper issues with how message state is managed.

2.3 Inspection Meetings

Code Inspection Meeting 1

- Date: 3/4/26
- Location: Ferland Room 202
- Time: 2:40p - 3:16p
- Participants:
    - Thread Zeppelin (Inspection Team)
    - PenUltimate(Development Team)

- Roles:
    - Recorder: Mariia Maksymenko
    - Inspectors: Ben Miller, Grace Murray
    - Moderator: Tess Nunes
    - Reader: Noah Rushing
- Discussed code and commenting conventions with PenUltimate along with security around passwords, requirements met and in progress, testing, and deployment

3. Modules Inspected

At the time of the code inspection, the IWAC Conference App had the majority of its functionality implemented. During the inspection meeting, PenUltimate indicated that the only major feature still in progress was the forgot-password functionality in the authentication flow. This feature allows users to request a password reset through an email link that redirects to the application. The team indicated that the feature was still being implemented and tested during the inspection and expected it to be completed by the end of the development cycle. The following sections describe the primary modules that were inspected.

3.1 Navigation and Application Layout Module

Files

- app/app/_layout.tsx
- app/app/(tabs)/_layout.tsx

Functionality

This module defines the primary navigation architecture of the app. It creates the application's root navigation stack and bottom tab navigation system using Expo Router. The root layout file (_layout.tsx) defines the stack navigator responsible for managing high-level application screens, including authentication screens, the main tab interface, conversation views, profile settings, and modal windows. The tab layout ((tabs)/_layout.tsx) defines the bottom navigation bar used throughout the application after login. The five main tabs include:

- My Agenda
- Sessions
- Messages
- Profile
- Notifications

Design Comparison and Tradeoffs

The SDD describes the application using a general MVC architecture but does not define navigation modules. The implemented system uses Expo Router to manage the stack navigator and tab navigation structure. This approach simplifies development because the routing logic is handled by the framework. As a result, the module primarily functions as part of the View layer.

3.2 Authentication Module

Files

- app/app/index.tsx
- app/app/createAccount.tsx
- app/lib/supabase.ts

Functionality

The authentication module controls user login and account creation. It allows new users to create accounts and returning users to authenticate using email and password credentials. The login screen validates user input and queries the users_registered table to verify that the user exists before calling the Supabase authentication API. If authentication succeeds, the user is redirected to the main application tabs. The create account screen performs validation on password length and password confirmation before creating a new account. The system then inserts records into the users and profiles tables and redirects the user into the application.

Design Comparison and Tradeoffs

The SDD describes user operations such as account creation and login through the User component of the system. In the implemented application these operations are performed through Supabase authentication APIs rather than custom logic. Using Supabase simplifies implementation and security.

3.3 Agenda and Session Management Module

Files

- app/app/(tabs)/index.tsx
- app/app/(tabs)/sessions.tsx
- app/components/panel-details.tsx

Functionality

This module manages conference sessions and the user's personal agenda. The Sessions tab retrieves all conference events from the database and allows users to search or filter sessions. Users may add sessions to their personal agenda. The My Agenda tab retrieves sessions that the user has saved in the user_agenda table and displays them in a list. Users can also remove sessions from their agenda. The panel-details.tsx component displays more detailed information about a session and allows users to perform actions such as adding the session to their agenda.

Design Comparison and Tradeoffs

The SDD describes presentation-related operations such as filtering and accessing presentation information. The implemented module retrieves events from the database and allows users to search and add sessions to their agenda. The module interacts with tables such as conference_events and user_agenda to store and retrieve data. One design difference is that the panel-details component performs direct database calls, which simplifies implementation but increases dependency between the UI and database.

3.4 Messaging Module

Files

- app/app/(tabs)/messages.tsx
- app/app/conversation.tsx

Functionality

The messaging module allows conference participants to send direct messages to other users. The messages.tsx screen displays conversation previews by retrieving messages where the current user is either the sender or recipient. The system then finds conversation partners and retrieves profile information to display conversation previews. The conversation.tsx screen displays messages exchanged between two users and allows users to send messages.

Design Comparison and Tradeoffs
The SDD defines a messaging capability through a sendMessage operation. The implemented messaging module is similar to this concept. It retrieves conversation previews and displays message histories between users. The system queries the messages table and retrieves profile information for conversation partners. This design improves the usability of the messaging interface but requires multiple database queries.

3.5 Profile Management Module

Files
- app/app/(tabs)/profile.tsx
- app/app/profilesettings.tsx

Functionality
This module allows users to view and modify profile information. The profile screen displays either the current user's profile or another user's profile depending on the situation. Profile information is retrieved from both the users and profiles tables. The profile settings screen allows users to update information such as first name, last name, profession, interests, and biography. Users may also upload a profile image.

Design Comparison and Tradeoffs
The SDD includes functionality for viewing and editing user profile information. The implemented module retrieves and updates profile data stored in the users and profiles tables. Profile updates and avatar uploads are handled through Supabase queries and storage services. The design is straightforward but includes minimal input validation, which may allow incomplete or incorrect profile data to be stored.

3.6 Notifications Module

Files
- app/app/(tabs)/notifications.tsx
- app/app/modals/notificationsettings.tsx

Functionality
The notifications module retrieves and displays notifications relevant to the current user. Notifications are retrieved from the user_notifications table and joined with the notifications table to produce a list of notification messages. The notification settings modal allows users to customize their preferences related to notifications.

Design Comparison and Tradeoffs
The SDD describes notification functionality for informing users of relevant events. The implemented module retrieves notifications from the user_notifications table and joins

them with the notifications table to display them. Notification preferences can be adjusted through the settings modal.

4. Defects

This section contains a list of actual defects discussed during the code inspection. The categories we used are defined as follows:

- *Correctness* with respect to the functional specification (does it look like it does what is required?)
- *Coding Convention* violation
- *Commenting* convention violation, missing or incorrect comments
- *User friendliness* (consider the handling of user input errors as well as prompts and all other information displayed to the user)
- *Other* (e.g., minimizing inter-module coupling, maximizing intra-module cohesion, refactoring)

Module	Description of the defect	Defect area
app/app/createAccount.tsx and app/app/(tabs)/profile.tsx	The sign-up validation checks for email presence, password length, and password confirmation, but it does not require first name or last name before account creation. We were able to create an account without those fields, which appears inconsistent with the intended profile data model, especially since the profile screen renders {profile.first_name} {profile.last_name}, and they would be blank if the user did not input them during the account creation.	Correctness with respect to the functional specification
app/app/createAccount.tsx	The create-account screen includes a primary button labeled “Login” even though the screen is primarily for account creation. We noticed that if a user chooses to create an account, the confirmation	User friendliness

	button still says “Login”. This is a minor defect, but it can be potentially misleading to users.	
app/app/index.tsx	The forgot-password flow is only partially implemented. In the code, pressing Forgot Password sends a reset email using a deep-link redirect (iwacapp://resetpassword), but during inspection it was reported that this feature did not fully work in practice and was still in progress. The implementation is also platform dependent.	Correctness with respect to the functional specification
app/app/modals/notificationsettings.tsx	The notification settings checkboxes appear to change only local UI state and do not persist settings or affect actual notification behavior. It was also reported that they cannot be tested on the web, which is a minor flaw right now.	Correctness with respect to the functional specification
app/components/panel-details.tsx	This shared component appears to contain direct Supabase/database calls. That creates tighter coupling between UI rendering and data access which makes the component harder to test in isolation.	Other (coupling / maintainability / testability)
app/app/(tabs)/messages.tsx and app/app/conversation.tsx	Any user can message any other user. This could be possibly okay with their client, but the lack of clearly stated restrictions may be a security concern, especially if messaging should be limited to mutual interactions only.	Other (security / design concern)
app/app/profilesettings.tsx	No protections were observed against inappropriate biography text or problematic	Other (security / content safety)

	uploaded profile images. Again, this is technically okay with their client, but it is a content-safety concern.	
app/app/createAccount.tsx, app/app/index.tsx, and other screens	Commenting conventions appear a bit inconsistent. PenUltimate stated that commenting was adopted more heavily only in the last few weeks. Some comments may appear above, below, or beside the code they describe, and some might have unnecessary symbols.	Commenting convention
app/app/createAccount.tsx, app/app/index.tsx, and other screens	Related to the above, several areas appear to have minimal explanatory comments relative to the amount of authentication, validation, and database logic being handled. And as stated above, they started commenting more heavily in the later stages of the development, so it appears they are making good progress to fix this issue.	Commenting convention
app/scripts/reset-project.js	The repository still contains the Expo scaffold reset script. The file itself states that it can be safely deleted after setup. Leaving it in the production repository is not recommended.	Other (refactoring / maintainability)
app/tests/ and related testing setup	The project appears to rely more heavily on local manual functional testing. This is not a major flaw as of right now, since teams are only starting the testing phase, and during the inspection we discussed their future testing plan.	Other (testing / maintainability)
repository assets / starter	Some unused starter assets	Coding Convention

files	such as default images (like React logo) appear to remain in the project. Deleting them would make the repository cleaner.	
app/app/(tabs)/index.tsx, app/app/(tabs)/sessions.tsx, and app/components/panel-details.tsx	Terminology appears inconsistent across the app and notes, with “event,” “session,” and “panel” sometimes used interchangeably. This is a minor flaw and should probably be discussed with their client.	User friendliness
app/app/profilesettings.tsx	The profile settings page allows editing first name, last name, profession, about me, interests, and my sessions, then writes them straight to Supabase. There is no visible validation for blank values, excessive length, inappropriate text, or file-type/size constraints on the avatar beyond image-picker selecting images.	User friendliness (improper user input validation)
app/app/(tabs)/sessions.tsx	In sessions.tsx, events are grouped with const key = r.session; and the grouped map is keyed only by the session string. If session labels can repeat across days, this could incorrectly combine events from different dates into one group.	Correctness

Appendix A – Team Review Sign-off

By signing this document, all parties agree they have reviewed the document and agree on its content and format. Any points of the document members may not agree with will be written in their personal comment section below.

Mariia:

Comments: N/A

3/11/26

Mariia Maksymenko Date

Benjamin:

Comments: N/A

3/11/26

Benjamin Miller Date

Grace:

Comments: N/A

3/11/26

Grace Murray Date

Tess:

Comments: N/A

3/11/26

Tess Nunes Date

Noah:

Comments: N/A

3/11/26

Noah Rushing Date

Appendix B – Document Contributions

Grace - Section 1, 1.1, 1.2, 2, Introduction (C+P), Appendices (C+P)
- 25%

Tess - Section 2.1, 2.3 (Description, Inspection Meeting)
- 15%

Mariia - Section 1.3 (Defect Checklist), 4 (Actual Defect List)

- 25%

Ben - Section 2.2
- 10%

Noah - Section 3 (Modules Inspected), 3.1, 3.2, 3.3, 3.4, 3.5, 3.6
- 25%